

Architecture rule: packet serialization and CRC-32 input must be deterministic. The sender and receiver must calculate CRC-32 over the same exact payload representation.

4. Technology Stack

Layer	Recommended technology	Purpose
Frontend	React / Next.js + HTML/CSS	Dashboard, controls, tables, charts
Realtime	WebSocket / Socket.IO	Live packet and KPI updates
Backend	Node.js + Express or Python FastAPI	APIs, simulation orchestration, analytics
CRC engine	Standard CRC-32 implementation/library	CRC generation and verification
Database	MongoDB	Runs, packets, sensor statistics
Charts	Chart.js / Recharts	Reliability and error trends
Simulator	Node.js or Python	101 sensors and packet generation

Recommended student stack: React/Next.js + Node.js/Express + MongoDB + a standard CRC-32 implementation. Python/FastAPI is an equally valid backend option.

No hardware dependency: the simulator replaces the role of real sensors and the communication channel, allowing the complete software workflow to be demonstrated without physical equipment.

5. Packet Format & CRC-32 Contract

Every packet must have a deterministic payload. The same exact bytes/string representation used to create the original CRC-32 must be used by the receiver when recalculating it.

Field	Example	Purpose
sensorId	S025	Identifies virtual sensor
sequenceNo	105	Orders packets; supports loss detection
temperature	28.5	Simulated temperature
humidity	72	Simulated humidity
timestamp	2026-09-04T14:30:25Z	Packet generation time
crc32	8F3A21C7	CRC-32 of payload

Recommended CRC input:

S025|105|28.5|72|2026-09-04T14:30:25Z

↓

CRC-32

↓

8F3A21C7

Keep delimiter, field order, numeric formatting, character encoding, and timestamp format identical at both ends. Otherwise the same displayed values can still produce different CRC values.

6. CRC-32 Validation Flow

CRC-32 is the core error-detection mechanism. It produces a 32-bit value derived from the packet payload. The receiver independently calculates CRC-32 and compares it with the value carried by the packet.

Sender: generate reading → build payload → calculate CRC-32(payload) → attach CRC → transmit.

Transmission: copy packet unchanged or modify it according to the configured corruption rule.

Receiver: read packet → separate payload and transmitted CRC → recalculate CRC-32 → compare → VALID or CORRUPTED.

VALID example:

```
Received CRC    = 8F3A21C7
Calculated CRC  = 8F3A21C7
Result          = VALID
```

CORRUPTED example:

```
Received CRC    = 8F3A21C7
Calculated CRC  = 2B91D440
Result          = CORRUPTED
```

Important: CRC-32 detects accidental data changes; it is not encryption, authentication, or a method for repairing corrupted data.

7. Sensor & Packet Simulation

The application creates 101 virtual sensors with IDs S001 through S101. Each sensor can generate readings at a configurable interval.

Parameter	Prototype setting	Notes
Temperature	20–40 °C	Random or patterned
Humidity	30–90 %	Random or patterned
Sequence number	Incrementing integer	One sequence per sensor
Timestamp	System time	Use consistent format

Simulation controls: sensor count (default 101), packets per sensor, packet generation rate, corruption probability, automatic/fixed-count mode, and optional random seed for repeatable tests.

8. Corruption & Packet-Loss Simulation

Since there is no physical network, transmission faults are represented by controlled software changes. The goal is repeatable CRC-32 testing rather than a complete wireless-network model.

Mode	Example	Expected result
Single-field change	28.5 → 38.5	CRC mismatch
Humidity change	72 → 73	CRC mismatch
Multiple-field change	Temperature + humidity	CRC mismatch
No corruption	Payload unchanged	CRC match
Packet drop (optional)	Sequence 104 absent	Loss if sequence tracking is enabled

Original payload:

S025|105|28.5|72|2026-09-04T14:30:25Z

CRC-32 = A1B2C3D4

Corrupted payload:

S025|105|38.5|72|2026-09-04T14:30:25Z

Received CRC = A1B2C3D4

Recalculated CRC = F72E91A8

Result = CORRUPTED

9. Reliability Metrics

The reliability engine aggregates packet results into metrics that can be understood during a project demonstration.

$$\text{Packet Reliability (\%)} = \frac{\text{Valid Packets}}{\text{Total Received Packets}} \times 100$$

$$\text{Error Rate (\%)} = \frac{\text{Corrupted Packets}}{\text{Total Received Packets}} \times 100$$

$$\text{Corruption Rate (\%)} = \frac{\text{Corrupted Packets}}{\text{Total Generated Packets}} \times 100$$

Metric	Example
Total generated	10,000
Valid	9,750
Corrupted	250
Reliability	97.5%
Error rate among received	2.5%

Sensor-wise results should show total, valid, corrupted, and reliability for each sensor. If packet loss is enabled, keep generated, received, valid, corrupted, and lost as separate categories.

10. Backend & Database

The backend controls simulation runs, exposes packet and analytics APIs, performs or delegates CRC-32 validation, and persists important results.

Collection	Important fields
SimulationRuns	runId, sensorCount, packetsPerSensor, corruptionRate, startTime, endTime, status
Sensors	sensorId, enabled, totalPackets, validPackets, corruptedPackets, reliability
Packets	packetId, runId, sensorId, sequenceNo, payload, originalCrc32, receivedCrc32, status, timestamp
NetworkStats	runId, generated, received, valid, corrupted, lost, reliability, errorRate

Core APIs: POST /api/simulation/start, POST /api/simulation/stop, POST /api/simulation/reset, GET /api/simulation/:id, GET /api/sensors, GET /api/packets, GET /api/analytics/summary, GET /api/analytics/sensors.

11. Web Dashboard / UX Requirements

The dashboard should feel like a monitoring console rather than a static report. Major simulation actions should produce visible feedback.

Component	Required information
Total Sensors	101 or configured count
Total Packets	Generated / received
Valid Packets	Count and percentage
Corrupted Packets	Count and percentage
Reliability	Overall percentage
Simulation Status	Idle / Running / Completed / Stopped
Corruption Rate	Current test rate

Simulation controls: sensor count, packets per sensor, packet rate, corruption probability, Start/Stop/Reset, and optional random seed.

Live packet monitor: show time, sensor ID, sequence number, transmitted CRC-32, recalculated CRC-32, and result without manual refresh.

Status language: use explicit labels VALID, CORRUPTED, RUNNING, IDLE; color may reinforce but must not be the only indicator.

12. Dashboard Pages / Screens

Screen	Main purpose
Dashboard	Network KPIs, reliability, current run
Simulation	Configure and run virtual sensor network
Packet Monitor	Inspect individual CRC validation results
Sensor Analysis	Compare all 101 sensors and identify high-error sensors
Run Details	Review one completed simulation and statistics

```

■ IoT SENSOR PACKET RELIABILITY MONITOR ■
■ Sensors ■ Packets ■ Valid ■ Corrupted■ Reliability ■
■ 101 ■ 10,000 ■ 9,750 ■ 250 ■ 97.5% ■
■ Reliability / Error Trend ■ Simulation Controls ■
■ chart ■ Sensors: 101 ■
■ Corruption: 5% ■
■ [ START ] [ STOP ] ■
■ Recent Packets: S025 105 A1B2C3D4 8F91D440 CORRUPTED ■

```

13. Error, Empty & Reliability States

Empty: “No simulation data yet — configure the test and start a run.” Do not display misleading 100% reliability when there are no packets.

Running: show live packet counts and an active indicator.

Completed: show final generated, received, valid, corrupted, lost, reliability, and highest-error sensors.

Simulator/backend failure: show a clear error and stop/pause safely; do not silently display stale results.

CRC mismatch: mark the packet CORRUPTED and retain the received payload and CRC information for analysis.

14. Reliability & Validation Rules

- CRC-32 input must be deterministic and identical at generation and verification time.
- Store transmitted/original CRC and recalculated CRC separately.
- A CRC mismatch always produces CORRUPTED.
- A CRC match means the packet passed the CRC check; it does not prove physical correctness or authenticity.
- Keep packet loss separate from CRC corruption if loss simulation is enabled.
- Every run must have a unique run ID.
- Reset must clearly separate old statistics from the new run.
- Handle empty datasets without division-by-zero or misleading metrics.

Test	Expected behavior
0% corruption	All received packets pass CRC-32
5% corruption	Approximately the configured fraction is corrupted
100% corruption	All altered packets fail CRC-32
One-field modification	Modified packet fails CRC-32
No data	Dashboard shows empty state
Reset	Counters and packet list return to initial state

15. Build Order

1. **Software skeleton:** frontend dashboard, backend API, database.
2. **CRC-32 engine:** implement and unit-test with fixed known test cases.
3. **One-sensor simulation:** prove generate → CRC → verify.
4. **Corruption simulator:** modify packets and confirm CRC mismatches.
5. **Scale to 101 sensors:** add scheduling and aggregation.
6. **Persistence:** store runs, packet results, and statistics.
7. **Live dashboard:** push packet events and counters without refresh.
8. **Analytics:** reliability trends and high-error ranking.
9. **Acceptance testing:** execute defined test cases and capture demo results.
10. **Stretch:** only after the core loop is stable.

16. Demo Script — What “Done” Looks Like

1. Open dashboard; show idle state.
2. Configure 101 sensors, packet count, and a corruption probability such as 5%.
3. Start simulation; show live packet generation and CRC-32 values.
4. Show a VALID packet where transmitted and recalculated CRC-32 values match.
5. Show a CORRUPTED packet where the payload changed and CRC values differ.
6. Show reliability and corrupted-packet counters updating.
7. Open Sensor Analysis and identify high-error sensors.
8. Complete the run and show final statistics.
9. Change corruption rate and run again to demonstrate different reliability behavior.

Final project definition: A software-based IoT communication reliability and data-integrity monitoring system that simulates 101 sensors, uses CRC-32 to detect simulated packet corruption, and visualizes reliability through a web dashboard.